IMPERIAL

Rotating Wave Approximation, Quantum Gates and the Quantum Fourier Transform

Will Noble
02651312
Group 3

Introduction to RWA

Quantum Computers require, at a minimum, two 1-Qubit logic gates and one 2-Qubit logic gate (Nielsen and Chuang 2010, p. 13). We will only be constructing 1-Qubit logic gates in this poster.

We want to introduce the Pauli matrices (Barnett 2026, p. 15), Physical Hamiltonian and Rotating Wave Approximation (RWA) (Barnett 2026, p. 18) to obtain quantum gates.

Pauli matrices:

$$\sigma_X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Physical Hamiltonian (Barnett 2026, p. 15):

$$\mathcal{H}_{exact}(t) = \frac{\epsilon}{2}\sigma_Z + \gamma\sigma_X \cos(\omega t)$$

In order to obtain the RWA, "it is customary to work entirely within the frame rotating at ω". (Barnett 2026, p. 18)

Rotating Frame:

$$|\psi'(t)\rangle = e^{i\frac{\omega t}{2}\sigma_Z}|\psi(t)\rangle$$

ROT/RWA:

$$\mathcal{H}_{rot}(t) = \frac{\epsilon - \omega}{2}\sigma_Z + \frac{\gamma}{2}\sigma_X + V(t)$$

$$V(t) = \frac{\gamma}{2}(\sigma_X cos(2\omega t) - \sigma_Y sin(2\omega t))$$

We then obtain $\mathcal{H}_{RWA} = \frac{\epsilon-\omega}{2}\sigma_Z + \frac{\gamma}{2}\sigma_X$ from dropping $V(t)$.

Is the RWA viable?

Is the RWA a close approximation to the Physical Hamiltonian when we drop the V(t)? This can be checked with an inner product.

In order to find the functions required for an inner product, we must solve the Time-dependent Schrödinger Equation (Barnett 2026, p. 9):

$$i\hbar\frac{\partial}{\partial t}|\psi(t)\rangle = \mathcal{H}(t)|\psi(t)\rangle$$

$\hbar = 1$ for all calculations in this poster.

We use:

$$|\psi(t)\rangle = |\psi(0)\rangle e^{-i\mathcal{H}(t)}$$

for our inner product, with $\mathcal{H}$ being picked as the Exact and RWA Hamiltonian for each function.

We use an inner product function on Python and compute the Fidelity (how close 2 Quantum operations are to each other (Ivezic 2023)) and Max Fidelity Error (M.F.E) for choices of $(\epsilon, \gamma, \omega)$. The Fidelity in this case is time-averaged state fidelity:

$$F_{\text{avg}} = \frac{1}{T}\int_0^T |\langle\psi_{exact}(t)|\psi_{RWA}(t)\rangle|^2 dt$$

The functions are compared on the closed interval $t \in [0, 10]$, using 10,000 data points.

Table 1: Fidelity and Max Fidelity Error (M.F.E). Obtained using NumPy (Harris et al. 2020) and SciPy (Virtanen et al. 2020) (trapezoid function).

ϵ	γ	ω	Fidelity	M.F.E
11	1	10	0.9980863	0.0066193
41	1	40	0.9998697	0.0004489
1001	1	1000	0.9999995	1.245E-6

When $\sqrt{(\epsilon-\omega)^2 + \gamma^2} \ll \omega$ is obeyed (which is required for negligible oscillations (Barnett 2026, p. 17)), the Exact and RWA Hamiltonian solutions to the Time-dependent Schrödinger Equation are very close, so we can drop V(t). This has been determined numerically rather than physical explanation.

Realised Single Qubit Gates

Using our Rotating Wave approximation, we must obtain realised gates of the form $U(\tau) = e^{-i\mathcal{H}_{\text{RWA}}\tau}$ (Barnett 2026, p. 20) and compare U to the Sought form of a gate. What parameters do we substitute in order to obtain the correct Quantum logic gates?

Table 2: Realised Gates Table

Gate	Sought Form	γ	τ	ϵ
Phase	$e^{-i\phi\sigma_Z}$	0	$\frac{2\phi}{\epsilon-\omega}$	$\epsilon \ll \omega$
Hadamard	$\frac{1}{\sqrt{2}}(\sigma_X + \sigma_Z)$	$\epsilon - \omega$	$\frac{\pi}{\sqrt{2}\gamma}$	$\epsilon \ll \omega$
X (NOT)	σ_X	$\neq 0$	$\frac{(2k+1)\pi}{\gamma}$	ω
$\sqrt{X}$	$\sqrt{\sigma_X}$	$\neq 0$	$\frac{(2k+1)\pi}{2\gamma}$	ω
$\frac{\pi}{8}$/T	$e^{-i\sigma_Z\frac{\pi}{4}}$	0	$\frac{2\phi}{\epsilon-\omega}$	$\epsilon \ll \omega$
S	$e^{-i\sigma_Z\frac{\pi}{2}}$	0	$\frac{2\phi}{\epsilon-\omega}$	$\epsilon \ll \omega$
$R_X(\theta)$	$e^{\frac{-i\sigma_X\theta}{2}}$	$\neq 0$	$\frac{\theta}{\gamma}$	ω
$R_Y(\theta)$	$e^{\frac{-i\sigma_Y\theta}{2}}$	$\neq 0$	$\frac{\theta}{\gamma}$	ω

$R_X(\theta), R_Y(\theta)$ are the Rotation gates with a respective axis X and Y.
Additionally, $U(\theta, \phi, \lambda)$ is the General Single Qubit Rotation Gate defined as $R_Z(\phi)R_Y(\theta)R_Z(\lambda)$, and picking $\phi = \frac{\theta}{2}$ yields $R_Z(\theta)$. $k \in \mathbb{Z}$ for the NOT and $\sqrt{X}$ gates.

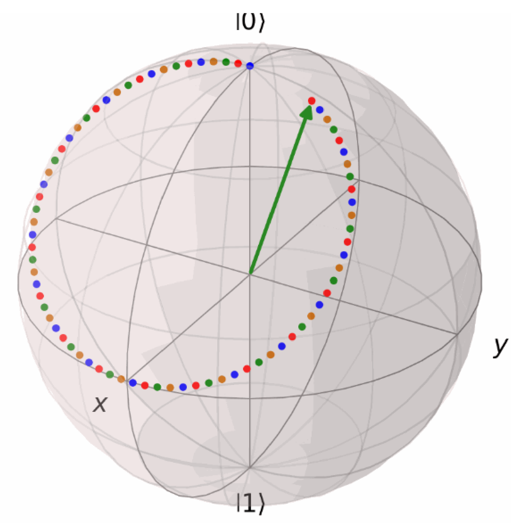
It is also very important to know The Dyadic Rational Phase Gate (Nielsen and Chuang 2010, p. 218),:

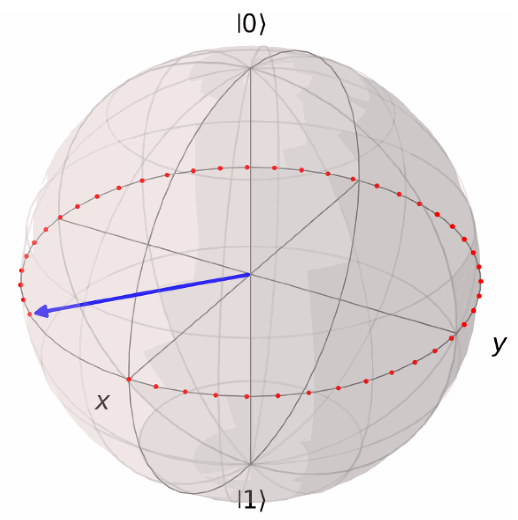
$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{\frac{2\pi i}{2^k}} \end{pmatrix}$$

We later use this gate for Quantum Fourier Transforms, with $R_1 = Z, R_2 = S, R_3 = \frac{\pi}{8}$/T gate.

The Bloch Sphere

It is also quite helpful to be able to visualise these gates, and a useful way of doing this is the Bloch Sphere. To get the idea, the σ_X, σ_Y and σ_Z Pauli Matrices are π radians anticlockwise rotations about their respective axis and the Phase gate is an anticlockwise rotation about the Z axis with an angle ϕ. Seen below is the Phase and Hadamard gates applied on $|0\rangle$ and $|+\rangle$ respectively. It is useful to know that the Hadamard has been applied twice, and it implies that $H^2 = I$.





These are produced as GIFs using Python's QuTip (Lambert et al. 2026), NumPy (Harris et al. 2020), ImageIO (ImageIO Contributors 2026) and OS packages.

Quantum Fourier Transforms

Now it seems realistic to go over an application of our Realised Logic Gates, yet we must include a 2-Qubit gate in order to do this. This gate is the SWAP gate, and is important in the application of the Quantum Fourier Transform (QFT).

We define the QFT by using the Discrete Fourier Transform. The QFT (Nielsen and Chuang 2010, p. 217) definition is seen below:

$$|j\rangle \equiv \frac{1}{\sqrt{N}}\sum_{k=0}^{N-1} e^{\frac{2\pi ijk}{N}}|k\rangle$$

Where N: Length of state, $|k\rangle$: a complex state of input data, $|j\rangle$: a complex state of transformed data.

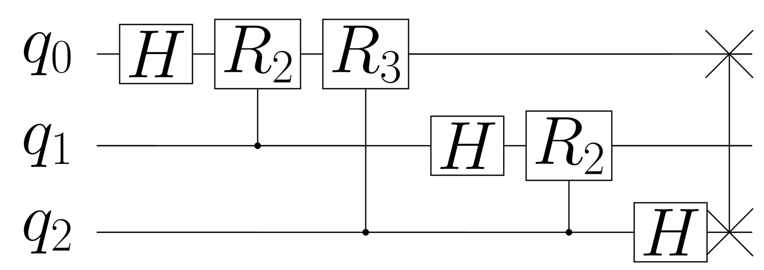
When computing a Quantum Fourier transform, we want to take $N = 2^n$ where n is the number of Qubits in our Quantum Computer.

Writing $|j\rangle = |j_1, ..., j_n\rangle$ in a binary representation, where $j = j_1 j_2 ... j_n$, we can eventually derive a product expression (Nielsen and Chuang 2010, p. 218) for the QFT using tensor operations:

$$|j\rangle = \frac{(|0\rangle + e^{2\pi i 0.j_n})(|0\rangle + e^{2\pi i 0.j_{n-1}.j_n}|1\rangle)...(|0\rangle + e^{2\pi i 0.j_1 j_2 ... j_n})}{2^{\frac{n}{2}}}$$

Where we use the notation $0.j_l j_{l+1}...j_m = \frac{j_l}{2} + \frac{j_{l+1}}{4} + ... + \frac{j_m}{2^{m-l+1}}$ which represents a binary fraction.

To see what this visually looks like for n = 3 (a 3-Qubit QFT), we present the following circuit:



This figure was created using Qiskit (Javadi-Abhari et al. 2024) and verified using NVIDIA's CUDA-Q v13.2 (NVIDIA 2026), applying a comparison against the QFT matrix that we apply on $|000\rangle$ to obtain our expected result (Qiskit and CUDA-Q are Python packages).

We can see the SWAP gate being used at the end of the QFT, where at most $\frac{n}{2}$ SWAP gates are required for a given QFT. It must also be noted that the initial state is represented as $|q_0 q_1 q_2\rangle$.

Quantum Fourier Transforms are immensely useful and it is much faster than computing an algorithm such as the Fast Fourier Transform (FFT), a classical algorithm (Nielsen and Chuang 2010, p. 220). Considering our current QFT circuit provided above, we have $\Theta(n^2)$ (considering the general form) compared against $\Theta(n2^n)$ from the FFT, showing that QFT is much faster, at best.

If I were to go further in researching this, it would be useful to consider Shor ($O(\log N^3)$) or Grover's ($O(\sqrt{N})$) algorithm, two popular quantum algorithms, used for factorising and searching for a short route respectively.

Citations, Derivations and Python